

WEEK 3 ENGINEERING REPORT

DATA PROFILING, QUALITY ISSUE VISUALIZATION, AND REPORTING

Author: R. Pooja Sekhar

Department: AI & Data Science

Institution: IIT Jodhpur

Domain: Industrial Python Programming

1. EXECUTIVE SUMMARY & OBJECTIVE

Data quality degradation is a primary vector of systemic failure in downstream analytical models, automated decision systems, and machine learning pipelines. Structural drift, unhandled null values, duplicate records, and statistical anomalies introduce severe operational friction and model bias. This engineering document establishes a comprehensive framework for automated data profiling and quality issue visualization using industry-standard Python libraries (pandas, matplotlib, seaborn). Designed as an end-to-end blueprint for Data Scientists and Data Engineers, this report defines rigorous profiling methodologies, automated detection algorithms, visual diagnostic strategies, and executive reporting outlines.

2. DATA PROFILING METHODOLOGY & TECHNICAL WORKFLOW

To establish an uncompromised foundation for exploratory data analysis (EDA) and model training, data profiling must proceed through a structured, reproducible engineering lifecycle across four distinct phases:

Phase 1: Metadata & Structural Inspection

Examine schema definition, data types, memory consumption, column non-null counts, and shape cardinality using pandas memory profiling tools.

Phase 2: Completeness & Null Analysis

Quantify missing value distributions across all attributes to distinguish between Missing Completely at Random (MCAR) and systematic failures.

Phase 3: Uniqueness & Cardinality Scans

Identify primary key collisions, duplicate row signatures, and categorical cardinality imbalances across dataset fields.

Phase 4: Statistical Outlier Isolation

Compute Interquartile Range (IQR) bounds and Z-score distributions to flag severe numerical anomalies and variance shifts.

3. DATA QUALITY ISSUES & VISUALIZATION FRAMEWORK

Effective communication of data quality defects to both technical and executive stakeholders requires tailored visual diagnostics:

QUALITY ISSUE	TECHNICAL DEFINITION & IMPACT	RECOMMENDED VISUALIZATION TOOL
1. Missing Values & Structural Gaps	Absence of critical attribute values leading to record dropouts, biased estimators, and model convergence failures during training.	Seaborn Missingness Matrix (msno.matrix) & Heatmap for pattern correlation.
2. Statistical Outliers & Anomalies	Numerical values falling outside standard distribution bounds (e.g., beyond 1.5 * IQR or 3 Z-scores), skewing parametric statistics.	Seaborn Box Plots & Violin Plots combined with Scatter Plots for multivariate isolation.

4. PYTHON IMPLEMENTATION GUIDE & CODE ARCHITECTURE

Below is an enterprise-grade Python module utilizing pandas, numpy, matplotlib, and seaborn to execute automated data profiling, missingness analysis, outlier detection, and visualization generation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

class DataProfiler:
    def __init__(self, filepath: str):
        self.filepath = filepath
        self.df = None

    def ingest_data(self):
        self.df = pd.read_csv(self.filepath)
        print(f"[INFO] Ingested {self.df.shape[0]} rows and {self.df.shape[1]} columns.")

    def profile_summary(self):
        print("--- Missing Values Summary ---")
        missing = self.df.isnull().sum()
        print(missing[missing > 0])
        print("\n--- Numerical Summary Statistics ---")
        print(self.df.describe())

    def detect_outliers_iqr(self, threshold=1.5):
        numeric_cols = self.df.select_dtypes(include=[np.number]).columns
        outlier_report = {}
        for col in numeric_cols:
            Q1 = self.df[col].quantile(0.25)
            Q3 = self.df[col].quantile(0.75)
            IQR = Q3 - Q1
            lower_bound = Q1 - (threshold * IQR)
            upper_bound = Q3 + (threshold * IQR)
            outliers = self.df[(self.df[col] < lower_bound) | (self.df[col] > upper_bound)]
            outlier_report[col] = len(outliers)
        return outlier_report

    def generate_visualizations(self):
        sns.set_theme(style="whitegrid")
        fig, axes = plt.subplots(1, 2, figsize=(14, 6))

        # Plot 1: Missing values bar chart
        missing_data = self.df.isnull().sum()
        missing_data = missing_data[missing_data > 0]
        if not missing_data.empty:
            missing_data.plot(kind='bar', ax=axes[0], color='#C0392B')
            axes[0].set_title("Missing Values per Feature", fontsize=12, fontweight='bold')
            axes[0].set_ylabel("Null Count")
        else:
            axes[0].text(0.5, 0.5, "No Missing Values", horizontalalignment='center')

        # Plot 2: Outlier box plot
        numeric_cols = self.df.select_dtypes(include=[np.number]).columns
        if len(numeric_cols) > 0:
            sns.boxplot(y=self.df[numeric_cols[0]], ax=axes[1], color='#2980B9')
            axes[1].set_title(f"Outlier Distribution: {numeric_cols[0]}", fontsize=12, fontweight='bold')

        plt.tight_layout()
        plt.savefig("data_quality_report_visuals.png", dpi=300)
        print("[INFO] Visualizations generated successfully.")
```

5. PROPOSED OUTLINE OF THE FINAL DATA QUALITY REPORT

- 1

Executive Summary & Business Risk Assessment
High-level overview of dataset health, quantified financial and operational risk exposure, and remediation urgency.
- 2

Dataset Architecture & Profiling Scope
Overview of data sources, ingestion volumes, schema definitions, and baseline descriptive statistics.
- 3

Comprehensive Data Quality Findings
Breakdown of missingness, duplicate rates, schema violations, and IQR outlier counts with embedded charts.
- 4

Root Cause Analysis (RCA) & Corrective Action Roadmap
Investigation into upstream injection failures, manual entry errors, automated imputation policies, and continuous monitoring alerts.

6. CONCLUSION & RECOMMENDATIONS

Automating data profiling and quality visualization is critical for maintaining high-integrity analytical pipelines and reliable machine learning models. By establishing programmatic checks using pandas, numpy, and advanced visualization libraries, data engineering teams can intercept anomalies before downstream propagation. It is strongly recommended to integrate the outlined profiling script into CI/CD data pipelines to ensure real-time data governance.